

Playbook — Fractional CTO

Guida operativa interna

Questo documento non è per il cliente. È per me. Scritto come se mi dessi istruzioni a me stesso il giorno in cui sto per sbagliare qualcosa di ovvio.

SEZIONE 1 — Il Ruolo: Cosa Fa Davvero un Fractional CTO

1.1 La distinzione che conta

Ci sono tre figure che il mercato confonde continuamente. Tenerle separate nella testa è il prerequisito per vendersi correttamente e per non finire a fare il lavoro sbagliato.

Figura | Focus | Orizzonte | Accountability

Fractional CTO | Direzione tecnica, strategia, leadership | 6-24 mesi | Business outcome

Consulente tecnico | Problema specifico, soluzione definita | Settimane/mesi | Deliverable

Staff augmentation | Capacità produttiva aggiuntiva | Sprint/mesi | Task completati

Il Fractional CTO è l'unico dei tre che risponde di risultati di business — non di codice consegnato, non di problema risolto, ma di "l'azienda è tecnicamente più solida e il team sa dove sta andando".

La confusione nasce perché spesso arrivo in aziende che non hanno mai avuto un CTO. Vogliono tutto. Il mio lavoro in fase di vendita è educare, non cedere.

1.2 La promessa al cliente

Non prometto "risolvo i problemi tecnici". Quello lo fa un consulente.

Prometto: garantisco la direzione tecnica dell'azienda per il periodo del contratto.

Tradotto in concreto:

- Il team sa cosa costruire e perché
- Le decisioni architetturali importanti vengono prese con metodo, non per istinto o urgenza
- Il CEO ha una vista chiara sullo stato tecnico e sui rischi
- Il team cresce — tecnicamente e come capacità di lavorare insieme

Questa promessa è verificabile. Alla fine del contratto o si è realizzata o no.

1.3 Le 4 responsabilità core

1. Strategia

Allineare la roadmap tecnica con gli obiettivi di business. Significa partecipare alle conversazioni strategiche dell'azienda, non solo a quelle tecniche. Se il CEO mi dice "vogliamo aprire il mercato tedesco tra 6 mesi", il mio lavoro è capire cosa significa tecnicamente e portare quel vincolo dentro la roadmap.

2. Architettura

Governare le decisioni strutturali: quali sistemi costruire, come si parlano, dove si accumula il debito tecnico, quando è il momento di riscrivere qualcosa. Non eseguo io l'architettura — la definisco, la valido, e insegno al team a ragionare in modo architettonico.

3. Team

Sviluppare la capacità tecnica del team. Il Tech Lead o Engineering Manager è il mio interlocutore primario. Con lui lavoro sulla sua crescita come leader tecnico. Con il team lavoro sulla cultura: come si prendono le decisioni, come si gestisce il debito, come si fa una retrospective utile.

4. Stakeholder

Tradurre il tecnico in business. Il board, gli investor, il CEO non tecnico — hanno bisogno di capire i rischi e le opportunità in termini di costo, velocità, e capacità di crescita, non in termini di microservizi e latenza.

1.4 Cosa NON faccio

Questo è importante quanto sapere cosa faccio. La lista va ribadita esplicitamente con il cliente durante l'onboarding e ogni volta che si inizia a scivolare.

- Non scrivo codice. Mai. Nemmeno "tanto per far vedere come si fa". Il momento in cui apro un IDE sto diventando un developer, non un CTO.
- Non gestisco ticket. Non sono in Jira a smistare task. Il backlog tecnico lo gestisce il team. Posso influenzarne le priorità in modo strategico, non operativo.
- Non sono il project manager. Non insegno le scadenze sprint. Non faccio status update quotidiani. Non sono il "responsabile dei delivery".
- Non sostituisco il Tech Lead. Se non c'è un Tech Lead, il mio lavoro è costruirne uno dall'interno — non fare io quel ruolo.
- Non partecipo a ogni meeting tecnico. Seleziono. Partecipo dove la mia presenza aggiunge valore strategico.

1.5 Script per comunicare queste differenze al cliente

Usare questo script nella prima call di vendita, quando il cliente mostra segni di confusione su cosa fa un Fractional CTO.

"Prima di andare avanti, voglio essere preciso su cosa faccio e cosa non faccio — mi aiuta a capire se siamo allineati.

Il mio ruolo è la direzione tecnica: strategie, architettura, team. Non scrivo codice, non gestisco sprint, non sono un tech lead affittato. La differenza pratica è che invece di risolvere i problemi tecnici per voi, vi costruisco la capacità di risolverli da soli, in modo coerente con dove volete andare.

Se stai cercando qualcuno che aumenta la produttività del team a breve termine, ho dei colleghi che fanno quello molto bene. Se stai cercando qualcuno che garantisce che le decisioni tecniche di oggi non diventino i problemi di scalabilità di domani, allora ha senso continuare a parlare."

Se il cliente, dopo questo script, continua a descrivere il ruolo in termini di task tecnici da eseguire, è un red flag qualificante. Non procedere.

SEZIONE 2 — Qualificazione Cliente

2.1 Il cliente ideale

Criteri positivi — tutti e tre devono essere presenti:

Criterio | Dettaglio | Perché conta

Stage | Scale-up 10-100 persone, Serie A/B o bootstrapped con traction | Troppo piccolo = non hanno budget. Troppo grande = hanno già un CTO.

Prodotto in produzione | Almeno un prodotto live con utenti reali | Senza prodotto in produzione, non c'è direzione tecnica da dare — c'è solo cos

Team tecnico presente | Almeno 3-5 developer, senza guida strategica | Il valore è moltiplicare il team esistente, non costruirlo da zero.

CEO/founder con gap | Non tecnico, o tecnico ma senza tempo/voglia di fare anche il CTO | Se il CEO vuole ancora fare il CTO, non c'è spazio per m

Profilo ideale in sintesi: Un founder che ha costruito un prodotto che funziona, ha assunto un team, ma sente che le decisioni tecniche avvengono per inerzia, che il tech debt si accumula senza un piano, che non riesce a capire se il team sta andando nella direzione giusta.

2.2 Red flag — non accettare

Questi sono criteri di esclusione. Se ne vedo uno, alzo la questione esplicitamente. Se ne vedo due, probabilmente declino.

Red flag | Segnale | Mia risposta

Vogliono un tech lead economico | "Abbiamo bisogno di qualcuno che guidi il team giorno per giorno e che faccia anche review del codice" | Chiarir

Nessun prodotto in produzione | "Siamo in fase di MVP" o "Stiamo costruendo la prima versione" | Potrebbe essere un engagement da consulente, non

Founder vuole "qualcuno che faccia anche il codice" | "Sarebbe bello se potesse darci anche una mano con qualche sprint" | Risposta netta: non lo

Budget <€5K/mese | Proposta a €3K-4K o "possiamo fare equity?" | Sotto €5K non ho le ore per fare il lavoro bene. Non accettare per poi essere fru

Cambi di priorità frequenti | Nella prima call cambia obiettivo due volte | Questo si gestisce (vedi Sezione 6), ma se già nella vendita è instabi

Mancanza di fiducia a priori | "Vogliamo un trial di un mese prima di impegnarci" | Un mese non è sufficiente per dare valore reale. Il minimum vi

2.3 Le 5 domande per qualificare in 20 minuti

Fare queste domande nell'ordine esatto. Le prime tre qualificano il problema. Le ultime due qualificano la maturità del cliente.

1. "Qual è la decisione tecnica più importante che avete rimandato negli ultimi 3 mesi?"

Se non sanno rispondere, o se la risposta è operativa (un bug, una feature), il problema strategico non è ancora percepito. Se sanno rispondere — e la risposta è architetturale o di team — il bisogno c'è.

2. "Com'è strutturato il team tecnico oggi? Chi prende le decisioni architetturali?"

Capisce la struttura reale e se c'è un Tech Lead. "Le decisioni le prendiamo insieme" o "le prende il senior developer" sono risposte che confermano l'assenza di governance tecnica.

3. "Qual è il vostro orizzonte temporale? Dove volete essere tra 12 mesi?"

Se non hanno una risposta, il CEO non sta facendo planning strategico. Se la risposta è solo di business ("vogliamo

raddoppiare i clienti") senza alcuna considerazione tecnica, confermano che hanno bisogno di qualcuno che costruisca quel ponte.

4. "Avete avuto incidenti tecnici negli ultimi 6 mesi? Come li avete gestiti?"

La risposta rivela la cultura tecnica esistente. "No" potrebbe essere positivo o potrebbe significare che non monitorano. "Sì, e abbiamo fatto una post-mortem blameless" = team maturo. "Sì, abbiamo punito chi ha fatto l'errore" = problema culturale serio.

5. "Perché ora? Cosa è cambiato rispetto a 6 mesi fa?"

Capisce il trigger. I trigger validi: crescita del team, nuovo round di funding, incidente grave, perdita del CTO precedente. Trigger meno robusti: "abbiamo letto di questo modello", "lo fa il nostro competitor". Non escludono, ma richiedono più validazione del problema percepito.

2.4 Matrice decisionale

Dopo le 5 domande, classifico:

Scenario | Decisione

Cliente ideale, budget ok, trigger chiaro | Proposta entro 48 ore

Cliente ideale, budget borderline (€5-6K) | Proposta con scope ridotto (8 ore/mese)

Buon problema, ma team troppo piccolo (<3 dev) | Rimando a 6 mesi, resto in contatto

Red flag su "tech lead economico" | Declino educatamente, suggerisco un Engineering Manager freelance

Nessun prodotto, ma founder credibile | Proposta da consulente strategico, non da Fractional CTO

Budget <€5K, non negoziabile | Declino. Non vale il tempo né per me né per loro.

SEZIONE 3 — Onboarding del Cliente (Settimane 1-4)

3.0 Pre-onboarding: cosa raccogliere prima del kick-off

Prima della prima settimana operativa, chiedere al cliente di preparare:

- [] Documento di tech stack (anche informale: linguaggi, framework, infra, cloud provider)
- [] Org chart del team tecnico (anche su un foglio di carta, fotografato)
- [] Roadmap attuale (anche solo "le prossime 3 cose che vogliamo costruire")
- [] Metriche baseline: deployment frequency, lead time, MTTR se esistono; altrimenti "come misurate la salute del team/prodotto?"
- [] Ultimi 3 incidenti o problemi significativi (data, descrizione, come è stato risolto)
- [] Accessi di lettura a: repository principale, issue tracker, dashboard di monitoraggio (se esiste)

Se non riescono a fornire queste cose, non bloccare l'onboarding — ma annotare cosa manca perché quella lista diventa già parte del gap da colmare.

3.1 Settimana 1 — Tech Assessment

Obiettivo: capire lo stato reale del sistema e del team, non quello percepito dal CEO.

Non è una code review. È un assessment di superficie che identifica i segnali di rischio e le aree da approfondire.

Cosa guardare nel codice/infrastruttura (2-3 ore di esplorazione):

Area | Cosa cercare | Segnale di rischio

Repository | Struttura, naming, presenza di test, ultima data di commit significativa | Nessun test, commit irregolari, nessun

README

CI/CD | Pipeline presente? Automatica? Quanto impiega? | Deploy manuali, nessuna pipeline, pipeline >30 min

Documentazione | ADR, README, diagram | Nessuna documentazione tecnica

Dipendenze | Versioni, aggiornamenti, vulnerabilità note | Dipendenze con anni di ritardo, CVE aperti

Configurazione | Secret in repo? Separazione ambienti? | .env committati, nessuna separazione dev/prod

Monitoraggio | Log, metriche, alert | Nessun logging strutturato, nessun alert

Domande per i developer (30 min a testa, 1-on-1):

- "Qual è la parte del sistema che ti preoccupa di più? Quella su cui non vorresti dover lavorare?"
- "Quando l'ultima volta hai deployato qualcosa e hai avuto paura che si rompesse? Cosa è successo?"
- "Se potessi cambiare una cosa nel modo in cui lavoriamo tecnicamente, cosa sarebbe?"
- "C'è qualcosa che tutti sanno ma nessuno dice ad alta voce?"
- "Chi è la persona che, se lasciasse domani, creerebbe il problema più grande?"

Domande per il CEO/founder:

- "Cosa ti aspetti da me nei prossimi 3 mesi? Come capisci se sta andando bene?"
- "Qual è il tuo livello di comfort attuale con le decisioni tecniche? Ti fidi del team?"
- "Ci sono tensioni nel team tecnico che devo sapere?"
- "Quali sono i 2-3 risultati di business che dipendono direttamente dalla tecnologia nei prossimi 6 mesi?"
- "C'è qualcosa che hai provato a fare tecnicamente e non ha funzionato? Perché pensi che sia successo?"

Come documentarlo:

Nessuna slide elaborata in settimana 1. Note grezze, organizzate per area. L'output finale è nella settimana 4.

3.2 Settimana 2 — Architecture Review (Quick Version)

Obiettivo: identificare top 5 rischi architetturali e top 3 opportunità. Non una review completa — quella richiede settimane. Questa è la vista di triage.

Durata: 4 ore totali (2 ore esplorazione, 2 ore sintesi).

Framework di analisi — guardare in questo ordine:

- Single points of failure: quali parti del sistema, se vanno giù, fermano tutto?
- Accoppiamento: dove i servizi/moduli sono così legati che cambiare uno richiede cambiare altri?
- Scalabilità bloccante: dove il sistema inizia ad avere problemi se gli utenti raddoppiano?
- Debito tecnico ad alto interesse: qual è il codice che rallenta ogni nuova feature?
- Gap di osservabilità: dove non saprei cosa sta succedendo in produzione?

Output settimana 2:

ARCHITECTURE QUICK REVIEW — [Nome Cliente]

Data: [data]

TOP 5 RISCHI (ordinati per impatto × probabilità)

1. [Rischio] — Impatto: Alto/Medio/Basso — Probabilità: Alta/Media/Bassa

Evidenza: [cosa ho visto]

Azione suggerita: [cosa fare]

2. ...

TOP 3 OPPORTUNITÀ

1. [Opportunità]

Prerequisiti: [cosa serve]

Stima beneficio: [cosa cambia]

2. ...

AREE DA APPROFONDIRE

- [Lista di cose che richiedono una review più profonda]

3.3 Settimana 3 — Team Assessment

Obiettivo: capire chi c'è, come lavora, dove sono i gap di leadership e competenza.

1-on-1 con ogni membro del team tecnico (30-45 minuti):

Non è un'intervista di performance. È una conversazione esplorativa. NLP attivo: ascolto, rispecchio, noto ciò che non viene detto.

Template 1-on-1 onboarding:

NOME: _____
 RUOLO: _____
 DATA: _____

APERTURA (5 min)

- Come stai? Come va il lavoro in generale?

CONTESTO (10 min)

- Da quanto sei qui? Come sei arrivato?
- Di cosa ti occupi principalmente?

TECNOLOGIA (10 min)

- Cosa ti piace di più del lavoro tecnico che fate?
- Cosa ti frustra di più?
- C'è qualcosa che vorresti imparare/fare che ora non fai?

TEAM E PROCESSO (10 min)

- Come descriveresti il modo in cui il team prende le decisioni tecniche?
- Cosa funziona bene nel processo? Cosa non funziona?

FUTURO (5 min)

- Dove ti vedi tra 12 mesi? Cosa vorresti che fosse diverso?

NOTE PRIVATE (non condividere):

- Energia/entusiasmo percepito (1-5):
- Livello tecnico percepito (1-5):
- Potenziale di leadership (1-5):
- Rischio di turnover (basso/medio/alto):
- Note qualitative:

Cosa sintetizzare dopo le 1-on-1:

- Chi è il leader informale del team (non sempre chi ha il titolo senior)
- Chi ha il rischio di turnover più alto
- Quali sono i temi ricorrenti nelle frustrazioni (segnali sistemici)
- Se esiste già un Tech Lead naturale o va costruito

3.4 Settimana 4 — Strategic Alignment

Obiettivo: allineare la roadmap tecnica con gli obiettivi di business per i prossimi 6-12 mesi.

Workshop con CEO + Tech Lead (2 ore):

Agenda:

- Revisione obiettivi di business (30 min) — cosa deve succedere nei prossimi 6/12 mesi?
- Vincoli tecnici su quegli obiettivi (30 min) — cosa ostacola o rallenta?
- Priorità tecniche conseguenti (30 min) — cosa va fatto prima?
- Decisioni da prendere insieme (30 min) — chi decide cosa, con quale criterio?

Output: lista di priorità tecniche condivise e validate dal CEO.

3.5 Output onboarding: "State of the Stack"

Il documento finale delle 4 settimane. È il contratto implicito tra me e il cliente su dove siamo e dove andiamo.

STATE OF THE STACK — [Nome Cliente]

Versione: 1.0 | Data: [data] | Preparato da: Elios Scoglio

EXECUTIVE SUMMARY

[3-5 righe: stato generale, messaggio principale, tono onesto]

1. TEAM

Struttura attuale: [org chart sintetica]

Punti di forza: [lista]

Gap: [lista]

Rischio turnover: [chi, livello]

Raccomandazione: [azione prioritaria]

2. ARCHITETTURA E SISTEMA

Stack attuale: [lista tech]

Stato CI/CD: [descrizione]

Stato osservabilità: [descrizione]

Top 5 rischi tecnici: [lista con impatto]

Top 3 opportunità: [lista]

3. PROCESSO

Come lavora il team: [descrizione]

Punti di forza del processo: [lista]

Inefficienze: [lista]

Raccomandazione: [azione prioritaria]

4. ALLINEAMENTO STRATEGICO

Obiettivi di business 6 mesi: [lista]

Roadmap tecnica prioritaria: [lista ordinata]

Decisioni pendenti: [lista]

5. PIANO AZIONE — PROSSIME 90 GIORNI

Priorità 1: [azione] — Owner: [chi] — Scadenza: [quando]

Priorità 2: ...

Priorità 3: ...

6. KPI DI RIFERIMENTO (BASELINE)

[Metriche misurabili con i valori attuali]

Deployment frequency: ____

Lead time for changes: ____

MTTR: ____

Team satisfaction (survey): ____

APPENDICE

- Note dettagliate assessment architetturale

- Note 1-on-1 (versione anonimizzata)

3.6 Checklist onboarding completa (50 item)

Pre-onboarding (prima del kick-off)

- ☐ Contratto firmato, prima fattura inviata
- ☐ Call di kick-off schedulata
- ☐ Richiesto tech stack document
- ☐ Richiesto org chart team tecnico
- ☐ Richiesta roadmap attuale
- ☐ Richiesti ultimi 3 incidenti/problemi
- ☐ Richiesto accesso read-only repository
- ☐ Richiesto accesso read-only issue tracker
- ☐ Richiesto accesso read-only monitoring/dashboard
- ☐ Intro email al team preparata (da far mandare dal CEO)

Settimana 1 — Tech Assessment

- ☐ Kick-off con CEO completato
- ☐ Intro al team completata
- ☐ 1-on-1 con ogni developer (almeno 30 min)
- ☐ 1-on-1 con CEO completata
- ☐ Repository principale esplorato
- ☐ CI/CD pipeline analizzata
- ☐ Documentazione tecnica esistente letta
- ☐ Dipendenze principali verificate (versioni, CVE)
- ☐ Configurazione e secret management verificati
- ☐ Stato monitoring e logging verificato
- ☐ Note assessment settimana 1 completate

Settimana 2 — Architecture Review

- ☐ Architecture review quick (4 ore) completata
- ☐ Top 5 rischi architetturali identificati
- ☐ Top 3 opportunità identificate
- ☐ Aree da approfondire documentate
- ☐ Draft quick review condiviso con Tech Lead per validazione

Settimana 3 — Team Assessment

- ☐ Template 1-on-1 onboarding compilato per ogni membro
- ☐ Leader informale identificato
- ☐ Rischio turnover valutato per ogni membro
- ☐ Temi ricorrenti sintetizzati
- ☐ Gap di competenza tecnica documentati
- ☐ Potenziali Tech Lead identificati (interni)

Settimana 4 — Strategic Alignment

- ☐ Workshop strategic alignment con CEO + Tech Lead completato
- ☐ Obiettivi di business 6/12 mesi documentati
- ☐ Vincoli tecnici su quegli obiettivi documentati
- ☐ Lista priorità tecniche condivisa con CEO
- ☐ Decisioni pendenti elencate con owner

Output finale

- ☐ "State of the Stack" documento completato
- ☐ KPI baseline stabiliti e documentati
- ☐ Piano azione 90 giorni condiviso con CEO
- ☐ Operating rhythm (4 momenti mensili) schedulato nel calendario
- ☐ Primo Strategic Planning mensile schedulato
- ☐ Canale di comunicazione principale definito (Slack/email/Teams)
- ☐ Escalation path definito (come mi contattano in urgenza)
- ☐ Accordo su frequenza check-in asincroni

SEZIONE 4 — I 4 Momenti Mensili (Operating Rhythm)

Ogni cliente attivo ha 4 momenti strutturati al mese. Totale: 7 ore di presenza diretta. Le ore rimanenti (fino a 12) vanno in preparazione, review asincrona, comunicazione scritta.

4.1 Strategic Planning (2 ore/mese)

Partecipanti: CEO + Tech Lead (eventualmente CTO se esiste nella transizione)

Agenda tipo:

Tempo | Contenuto

00:00-00:15 | Review mese precedente: cosa era previsto, cosa è successo, perché
 00:15-00:45 | Stato roadmap tecnica: avanzamento, blocchi, cambiamenti di priorità
 00:45-01:15 | Decisioni tecniche strategiche da prendere questo mese
 01:15-01:45 | Aggiornamento rischi (nuovo, cambiato, risolto)
 01:45-02:00 | Prossimi passi, owner, scadenze

Preparazione (30 min prima della call):

- Leggere il report mensile del mese precedente
- Controllare il backlog tecnico: ci sono decisioni architetturali pendenti?
- Verificare lo stato degli incident del mese
- Preparare 1-3 punti che voglio portare io, non solo rispondere a quelli del cliente
- Aggiornare la lista rischi

Output della sessione:

- Decisioni prese (scritte, non solo verbali)
- Roadmap aggiornata con priorità riviste
- Lista rischi aggiornata
- Azioni specifiche con owner e scadenza

Regola: ogni decisione presa in questa sessione va scritta entro 24 ore in un documento condiviso. Se non è scritta, non è una decisione.

4.2 Architecture Review (2 ore/mese)

Partecipanti: Tech Lead + eventualmente i senior developer

Obiettivo: non trovare bug o fare code review. Verificare che le decisioni architetturali del mese siano state prese con consapevolezza dei trade-off.

Cosa guardare ogni mese:

Area | Domande

Nuove feature | Hanno introdotto accoppiamento nuovo? C'è debito deliberato?
 Tech debt | Il debito si sta accumulando o stiamo pagando qualcosa?
 Incidenti del mese | Cosa ci hanno insegnato? C'è un pattern sistemico?
 Decisioni tecniche prese | Erano le migliori? Sono documentate?
 Dipendenze esterne | Nuove dipendenze? Rischi di aggiornamento?

Come dare feedback costruttivo senza demotivare:

La distinzione che fa la differenza: feedback sull'architettura (il sistema), non sul developer (la persona).

Invece di: "Questa scelta era sbagliata"

Dire: "Questa scelta ha senso per il problema che avevate allora. Guardando avanti, c'è un trade-off che vale la pena valutare..."

NLP applicato: usare il linguaggio "noi" quando si parla di problemi del sistema. Usare il linguaggio "tu" quando si parla di punti di forza individuali.

Quando escalare a una full architecture review:

- Un incidente grave ha rivelato un problema strutturale
- Il team sta pianificando una riscrittura o un cambiamento architetturale significativo
- La performance o la scalabilità sta diventando un problema concreto
- Si sta valutando una fusione/acquisizione o una partnership tecnica

Una full architecture review richiede 2-3 giorni di lavoro dedicato, non 2 ore. Va pianificata e fatturata separatamente.

4.3 Team Mentoring (2 ore/mese)

Partecipanti: Tech Lead o Engineering Manager (raramente altri)

Nota: non faccio 1-on-1 con tutti i developer ogni mese. Quella è responsabilità del Tech Lead. Il mio focus è sviluppare il Tech Lead come leader tecnico.

Struttura della 1-on-1 mensile con il Tech Lead:

Fase	Tempo	Contenuto
Aggiornamento	20 min	Come va? Cosa è successo? Cosa ha funzionato, cosa no?
Sfida corrente	25 min	Qual è la cosa più difficile che stai affrontando? Come la stai gestendo?
Sviluppo	10 min	Cosa stai imparando? Dove vuoi crescere?
Azioni	5 min	Cosa farà lui prima della prossima call? Cosa faccio io per supportarlo?

Come gestire il caso in cui non c'è un Tech Lead:

Se il team non ha un Tech Lead, il mio obiettivo è identificare e sviluppare la persona giusta entro 3-6 mesi. Nel frattempo:

- Scelgo la persona con il maggior potenziale di leadership (dalla mia analisi onboarding)
- La coinvolgo nelle Architecture Review
- La includo nello Strategic Planning (non come partecipante passivo — come contributor)
- Le assegno responsabilità crescenti e la supporto nel gestirle

Se dopo 6 mesi non emerge nessuno internamente, raccomando al cliente di assumere un Engineering Manager. Non è il mio ruolo farlo io a lungo termine.

Secondo slot disponibile (1 ora/mese):

Se necessario, posso avere una seconda sessione di mentoring mensile con un developer chiave (tipicamente uno che sta crescendo verso il ruolo di Tech Lead, o uno a rischio di turnover che vale la pena trattenerlo). Da usare con parsimonia.

4.4 Stakeholder Update (1 ora/mese)

Partecipanti: CEO (obbligatorio). Board/investor se presenti.

Obiettivo: dare al CEO una vista chiara sullo stato tecnico e rimuovere l'ansia dell'ignoto.

Il report mensile — struttura 1 pagina:

TECH UPDATE — [Nome Cliente] — [Mese Anno]

HIGHLIGHT DEL MESE

- [Cosa è andato bene tecnicamente]
- [Cosa abbiamo migliorato]
- [Una decisione importante presa]

RISCHI ATTIVI

Rischio	Livello	Azione in corso	Owner
-----	-----	-----	-----
...	Alto	...	...

DECISIONI RICHIESTE AL CEO

1. [Decisione] — Entro: [data] — Perché ora: [motivo]

PROSSIMO MESE

- Focus tecnico principale: [cosa]
- Milestone attesa: [cosa]
- Dipendenze critiche: [cosa serve da chi]

METRICHE CHIAVE

Deployment frequency: [valore] [trend ↑↓→]
 Incidenti: [numero] [vs mese precedente]
 [Altra metrica rilevante per questo cliente]

Come parlare con un CEO non tecnico:

Regola base: ogni affermazione tecnica va seguita immediatamente dall'impatto di business.

Invece di: "Abbiamo identificato un problema di N+1 query nel modulo ordini"

Dire: "C'è un'inefficienza nel codice degli ordini che, man mano che cresciamo, rallenterà le pagine di acquisto. Non è urgente oggi, ma lo diventa prima che raggiungiamo i [X] utenti attivi."

Mappa di traduzione sempre utile:

Termine tecnico | Traduzione business

Debito tecnico | Costo nascosto di velocità futura

Incidente di produzione | Il prodotto non era disponibile per [X] minuti, impatto su [Y] clienti

Scalabilità | Quanti utenti/transazioni possiamo gestire prima che si rompa

Refactoring | Manutenzione necessaria che non aggiunge funzionalità ora, ma accelera tutto dopo

CI/CD | Quanto velocemente possiamo mettere qualcosa in produzione in sicurezza

Template board update (per clienti con investor board):

ENGINEERING UPDATE — [Trimestre]

STATUS: [Verde/Giallo/Rosso] — [Una riga di sintesi]

KEY ACHIEVEMENTS

- [Achievement 1 con metrica]
- [Achievement 2 con metrica]
- [Achievement 3 con metrica]

RISKS & MITIGATIONS

Risk	Impact	Likelihood	Mitigation
-----	-----	-----	-----
...	High	Medium	...

METRICS

[Solo 4-6 metriche con trend. Niente di più.]

NEXT QUARTER PRIORITIES

1. [Priorità con obiettivo misurabile]
2. ...
3. ...

INVESTMENT REQUIRED

[Solo se serve decisione del board: budget, hiring, strumenti]

SEZIONE 5 — Governance Tecnica

5.1 ADR — Architecture Decision Records

Quando creare un ADR:

- Decisione che impatta più di un servizio/componente
- Decisione difficile da invertire nel breve termine
- Decisione su cui il team non è unanime
- Scelta di tecnologia o framework nuovo
- Decisione che tocca sicurezza, compliance, o dati

Quando NON creare un ADR:

- Decisioni di implementazione locale (come strutturare un file, quale nome dare a una variabile)
- Decisioni temporanee e reversibili senza costo significativo

Template ADR:

```
# ADR-[NNN]: [Titolo della decisione]

**Data:** [YYYY-MM-DD]
**Status:** [Proposta / Accettata / Deprecata / Sostituita da ADR-NNN]
**Deciders:** [Chi ha partecipato alla decisione]

## Contesto
[Qual era la situazione che richiedeva una decisione? Quali vincoli c'erano?]

## Decisione
[Cosa abbiamo deciso. Frase affermativa, non "abbiamo valutato".]

## Opzioni considerate
1. [Opzione A] — Pro: [...] — Contro: [...]
2. [Opzione B] — Pro: [...] — Contro: [...]
3. [Opzione C] — Pro: [...] — Contro: [...]

## Conseguenze
**Positive:** [cosa migliora]
**Negative:** [cosa peggiora o cosa dobbiamo accettare]
**Rischi:** [cosa potrebbe andare storto]

## Note
[Riferimenti, link, commenti aggiuntivi]
```

Dove tenerli:

- Prima scelta: repository principale, cartella /docs/adr/
- Alternativa: Notion con database dedicato (se il team non usa git per la documentazione)
- Mai: email, Slack, Google Docs sparsi

Il mio ruolo sugli ADR:

- Non li scrivo io (li scrive il Tech Lead o il developer che ha proposto la decisione)
- Li reviso e li firmo come approvazione quando la decisione è strategica
- Li uso come strumento di sviluppo del Tech Lead: "prima di implementare, scrivimi un ADR"

5.2 Tech Radar

Cos'è: lista delle tecnologie/pratiche divise in 4 anelli: Adotta, Valuta, Sospendi, Abbandona.

Come costruirlo con il team (workshop 2 ore, una volta ogni 6-12 mesi):

- Raccogliere input in anticipo: chiedere a ogni membro del team di nominare 3-5 tecnologie/pratiche da posizionare
- Workshop: mettere i post-it su una board virtuale (Miro/Mural) nei 4 anelli
- Discutere le divergenze — dove non siamo d'accordo è dove c'è più valore
- Consolidare e pubblicare

Non è un documento di regole. È uno strumento di conversazione. Il valore è nel processo, non nel risultato.

Il mio ruolo:

- Facilitare il workshop
- Contribuire con pattern che vedo in altri contesti
- Prevenire sia il conservatorismo ("non cambiamo mai nulla") che la mania di novità ("adottiamo tutto ciò che è uscito nell'ultimo anno")

5.3 Hiring Review

Il mio ruolo nel processo di assunzione tecnico:

- Definire il profilo tecnico con il CEO e il Tech Lead
- Partecipare al technical interview finale (non al primo screening)
- Dare feedback tecnico scritto dopo il colloquio
- Non essere il recruiter, non fare sourcing, non fare il primo screening

Cosa valuto nel colloquio tecnico:

Non valuto la conoscenza delle ultime tecnologie. Valuto:

- Come ragiona di fronte a un problema che non conosce
- Come descrive un suo errore passato e cosa ha imparato
- Come spiega un concetto tecnico a qualcuno non tecnico
- Come reagisce quando gli faccio notare una falla nel suo ragionamento

Segnali di non-fit che il team tende a non vedere:

- "Faccio meglio da solo" (red flag di collaborazione)
- Risponde a ogni domanda con la stessa tecnologia/framework (pensiero rigido)
- Non sa nominare un progetto o una decisione di cui si è pentito (mancanza di riflessività)

5.4 Sprint/Cycle Review

Come partecipare senza diventare un'altra figura di governance che il team sente come peso:

- Partecipo a max 1 sprint review al mese per cliente
- Non do feedback sulle singole task — do feedback sul pattern complessivo
- Se noto un problema sistemico, ne parlo con il Tech Lead nella 1-on-1, non durante la review davanti al team

Cosa osservo durante una sprint review:

- Il team capisce il valore di business di quello che ha consegnato?
- Le demo sono di qualità? Il team è orgoglioso di quello che mostra?
- Come gestiscono le cose non completate? Con accountability o con giustificazioni?
- C'è psychological safety — le persone si esprimono o aspettano che parli sempre il senior?

5.5 Incident Post-Mortem

Come facilitare una blameless retrospective:

Il blameless post-mortem è uno degli strumenti dove il mio background NLP è più utile. Le persone arrivano con difese alte perché hanno paura di essere incolpate.

Struttura (90 minuti):

Fase | Tempo | Focus

Timeline degli eventi | 20 min | Ricostruzione fattuale, cronologica, senza giudizi

Analisi delle cause | 30 min | "Cosa ha reso possibile questo?" non "Chi ha fatto questo?"

Impatto | 10 min | Utenti/business impattati, durata, costo stimato

Azioni correttive | 20 min | Cosa cambiamo nel sistema, nel processo, negli strumenti

Learning | 10 min | Cosa ha funzionato bene anche nell'incidente?

Regole della facilitazione:

- Ogni volta che qualcuno dice "lui/lei ha fatto X" — riformulare: "il sistema/processo ha permesso che X accadesse"
- Se si cerca un colpevole, interrompere educatamente: "Capisco la frustrazione. Il nostro obiettivo ora è che non si ripeta, non trovare chi punire. Se puniamo, la prossima volta qualcuno nasconde l'incidente."
- Concludere sempre con: "Cosa ha funzionato bene? Cosa ha limitato il danno?"

Output obbligatorio del post-mortem:

- Documento scritto con timeline, cause, azioni
- Owner e scadenza per ogni azione
- Pubblicazione interna al team (la trasparenza costruisce fiducia)

SEZIONE 6 — Gestione di Situazioni Difficili

6.1 Cliente che vuole micro-gestire il mio lavoro

Segnali: mi chiede report settimanali dettagliati di cosa ho fatto, vuole essere invitato a tutti i meeting, mette in discussione ogni mia raccomandazione prima di darle il tempo di funzionare.

Root cause tipica: paura di non avere controllo su una funzione che non capisce del tutto. Non è cattiveria — è ansia.

Come gestirlo:

- Non resistere direttamente — la resistenza aumenta l'ansia.
- Aumentare la trasparenza proattiva — se mi chiede più report, do report più frequenti ma più brevi. L'obiettivo è che smetta di chiedere perché riceve già tutto quello che gli serve.
- Nominare il pattern (in una conversazione 1-on-1, non per email): "Noto che hai bisogno di più visibilità su quello che sto facendo. Voglio capire cosa manca perché tu ti senta sicuro. Cosa dovrei darti che ora non ti do?"
- Definire un SLA di risposta per le sue domande: "Se mi mandi un messaggio, rispondo entro [X ore]. Non mi interrompere durante le sessioni con il team."

Se non migliora dopo 2 mesi: conversazione diretta. "Il modo in cui stiamo lavorando non è sostenibile per nessuno dei due. Ho bisogno di operare con autonomia all'interno del perimetro che abbiamo definito insieme, altrimenti non riesco a darti il valore per cui sono qui."

6.2 Team che non mi accetta

Segnali: risposte monosillabiche nelle 1-on-1, non ricevo feedback onesto, nelle Architecture Review le persone concordano con tutto senza pensarci.

Root cause tipica: il team ha già visto "consulenti" arrivare, fare slide, e andarsene lasciando un disastro. Hanno imparato a proteggersi.

Come costruire fiducia (non si forza, si guadagna):

- Essere utile prima di essere autorevole — il primo mese aiuto il team a risolvere problemi concreti, senza giudicare.
- Non portare risposta ai problemi nelle prime settimane — portare domande. Fare sembrare che io stia imparando da loro (perché è vero).
- Riconoscere pubblicamente il buon lavoro — ma solo quando è reale. I complimenti falsi si sentono.
- Ammettere quando non so qualcosa — il team rispetta l'onestà molto più della competenza performata.
- Mantenere le promesse piccole — se dico "ti mando quella documentazione entro venerdì", la mando entro venerdì.

Timeline realistica: 4-8 settimane per le basi. 3-6 mesi per una fiducia profonda.

Se non migliora: capire se c'è un blocco specifico. Spesso c'è una persona che ha un'influenza informale sul gruppo e che non è convinta. Una conversazione diretta con lei può sbloccare tutto il resto.

6.3 CEO che cambia priorità ogni settimana

Segnali: ogni call parte da una direzione diversa. La roadmap tecnica cambia ogni 2 settimane. Il team è frustrato e

disorientato.

Root cause tipica: il CEO non ha un framework di prioritizzazione. Reagisce all'input più recente (ultimo cliente insoddisfatto, ultimo competitor analizzato, ultima conversazione con un investor).

Come stabilizzare:

- Introdurre un ciclo di pianificazione — lo Strategic Planning mensile diventa l'unico momento in cui si cambia la direzione tecnica. Fuori da quel momento, il cambio richiede una conversazione esplicita.
- Nominare il costo del cambio — ogni volta che il CEO propone un cambio di priorità fuori ciclo: "Possiamo farlo. Il costo è [X settimane di ritardo su Y]. Vuoi procedere lo stesso?"
- Costruire uno stack rank — mai lista di priorità non ordinata. Sempre: "Queste sono le 5 priorità. Quale è la numero 1 se potessimo fare solo una?"
- Usare il NLP — il CEO che cambia priorità spesso è un CEO che non si sente ascoltato. Prima di contrastare il nuovo cambio: "Cosa ti ha fatto pensare a questo? Cosa stai cercando di risolvere?" Spesso il problema reale è diverso da quello dichiarato.

6.4 Incidente critico — cosa fare nelle prime 2 ore

Questa sezione esiste perché se si verifica un incidente critico mentre sono ingaggiato, il cliente si aspetta che io sappia cosa fare.

Minuto 0-15: Stabilizzare

- Capire l'impatto: quanti utenti colpiti? Cosa non funziona esattamente?
- Chi è on-call? Se non c'è nessuno definito, chi è disponibile ora?
- Non stare a fare root cause analysis — prima ripristinare il servizio.

Minuto 15-60: Comunicazione

- CEO informato (anche solo un messaggio: "Siamo a conoscenza del problema, stiamo lavorando, aggiornamento tra 30 minuti")
- Se ci sono clienti impattati, chi comunica loro? (non necessariamente io — ma qualcuno deve)
- Aggiornamenti frequenti anche senza novità: "Ancora in corso, causa non trovata, stimiamo X minuti"

Ora 1-2: Risoluzione o workaround

- Se la causa non è trovata entro 1 ora, valutare rollback alla versione precedente
- Documentare tutto quello che si fa, in tempo reale (Slack/Note condiviso)
- Non tentare fix multipli simultanei — uno alla volta, con verifica

Dopo l'incidente:

- Post-mortem entro 48 ore
- Comunicazione conclusiva a tutti gli stakeholder
- Azioni correttive con owner e scadenza

6.5 Quando terminare il contratto

Criteri oggettivi per terminare:

Situazione | Soglia | Azione

Il cliente non implementa nessuna raccomandazione strategica | 3 mesi consecutivi | Conversazione diretta, poi uscita se non cambia

Il budget viene tagliato sotto la soglia minima | <€5K/mese | Rinegoziare scope o uscire

Cambio di CEO o di strategia che rende il mio ruolo non necessario | Valutazione caso per caso | Exit pianificato

Conflitto di valori irreparabile | Caso per caso | Uscita rapida ma professionale

Il cliente inizia ad assumere un CTO full-time | Successo — transizione pianificata | Exit professionale, supporto nella transizione

Come farlo:

- Mai di sorpresa — segnali di pre-uscita almeno 4-6 settimane prima
- Conversazione diretta prima della comunicazione formale: "Voglio essere onesto su dove siamo. Ho l'impressione che il mio ruolo non stia creando il valore che entrambi ci aspettavamo. Parliamone."
- Exit plan scritto — chi prende in carico le attività in corso, stato dei progetti, documentazione da completare
- Fine professionale — non bruciare la relazione. Il mercato è piccolo.

Script per la conversazione di uscita:

"Ho riflettuto sul nostro engagement. Penso che siamo in un momento in cui [situazione oggettiva]. Il mio ruolo così com'è strutturato non è più quello giusto per voi. Posso immaginare due strade: [Opzione A — rivedere il contratto] o [Opzione B — concludere con una transizione pianificata]. Quale preferisci esplorare?"

6.6 Il cliente non implementa le raccomandazioni

Questo è il caso più frustrante e più comune. Ho fatto l'analisi, ho dato le raccomandazioni, il mese dopo siamo nello stesso posto.

Prima di reagire, capire il perché:

Causa | Soluzione

Non hanno le risorse (tempo/persona) per implementare | Aiutare a prioritizzare: "Se potete fare solo una cosa, quale ha il maggior impatto?"

Non condividono davvero la raccomandazione | Riaprire la conversazione: "Noto che questa cosa non è avanzata. C'è qualcosa che non ti convince?"

La raccomandazione non era sufficientemente concreta | Riformulare come piano d'azione con step espliciti e owner

Il CEO ha altre priorità non dichiarate | Conversazione diretta sul contesto reale

Quando diventa un problema di contratto:

Se dopo 3 mesi nessuna raccomandazione strategica viene implementata, sto prendendo soldi per scrivere documenti che nessuno legge. Non è sostenibile. Conversazione diretta, come sopra.

SEZIONE 7 — Scalabilità: Gestire 3-4 Clienti Simultanei

7.1 Time-boxing per cliente

Massimo per cliente standard: 12 ore/mese.

Allocazione standard:

Attività | Ore/mese

Strategic Planning (sessione) | 2.0

Architecture Review (sessione) | 2.0

Team Mentoring (sessione) | 2.0

Stakeholder Update (sessione) | 1.0

Preparazione (tutte le sessioni) | 2.0

Comunicazione asincrona (Slack/email) | 1.5

Report mensile e ADR review | 1.5

Totale | 12.0

Clienti premium (€15K+/mese): fino a 20 ore/mese — aggiungere sprint review partecipazione, hiring interview, full

architecture review trimestrale.

7.2 Il calendario settimanale tipo con 3 clienti attivi

Principio: non mischiare i contesti nella stessa giornata. Un giorno = un cliente (o lavoro non-cliente).

Struttura settimana tipo:

Giorno | Focus | Attività

Lunedì | Cliente A | Sessione mensile o prep

Martedì | Cliente B | Sessione mensile o prep

Mercoledì | Cliente C | Sessione mensile o prep

Giovedì | Sviluppo business + admin | Chiamate prospect, fatturazione, aggiornamento playbook

Venerdì | Deep work personale | Studio, writing, formazione, Yoga

Regole:

- Le sessioni con i clienti: solo lunedì-mercoledì, mai venerdì
- Il giovedì non è un giorno di client delivery
- Il venerdì è sacro — nessun cliente salvo emergenza

Con 4 clienti: spostare uno dei clienti al giovedì mattina, mantenere il pomeriggio libero per admin.

7.3 Come usare l'AI per accelerare il lavoro

L'AI è un moltiplicatore di produttività, non un sostituto del pensiero. Usarla per il lavoro ripetitivo, non per il lavoro strategico.

Dove l'AI aiuta davvero:

Task | Come usarlo

Analisi codice | Incollare snippet e chiedere "quali rischi architetturali vedi?" — poi validare con il mio giudizio

Bozza report mensile | Dare bullet point, chiedere stesura in formato template — poi riscrivere le parti che richiedono giudizio

Preparazione 1-on-1 | "Data queste note della 1-on-1 precedente, quali domande devo fare questa volta?"

ADR bozza | "Data questa decisione [X], scrivimi una bozza di ADR" — poi revisionare

Ricerca tech | "Confronta [tecnologia A] e [tecnologia B] per questo use case" — da usare come input, non come conclusione

Post-mortem template | Strutturare la timeline e l'analisi delle cause

Dove l'AI non sostituisce il mio giudizio:

- La valutazione delle persone (Tech Lead assessment, hiring)
- Le raccomandazioni strategiche ad alto impatto
- La gestione delle relazioni con CEO e board
- La lettura del contesto politico/emotivo del cliente

7.4 Quando assumere un junior consultant per supporto

Soglia: quando supero i 3 clienti e sento che la qualità sta scendendo, o quando trascorro >30% del tempo su task operativi (preparazione documentazione, update report, ricerca tecnica).

Profilo: non un developer. Un business analyst o un recente consultant con background tecnico. Il loro compito è ridurre il mio overhead operativo, non fare le parti strategiche.

Task delegabili:

- Prima stesura report mensili
- Ricerca su tecnologie/pattern specifici

- Strutturazione documentazione ADR
- Monitoraggio metriche e aggiornamento dashboard
- Coordinamento logistico meeting

Task non delegabili:

- Sessioni con CEO e board
- Raccomandazioni strategiche
- 1-on-1 con Tech Lead
- Architecture Review

7.5 Segnali di burnout e come prevenirli

Con 3-4 clienti, il burnout non è una possibilità teorica. È un rischio operativo da monitorare come si monitora la salute di un sistema.

Segnali precoci (intervenire subito):

- Ogni domenica sera mi sento ansioso rispetto alla settimana che inizia
- Sto rimandando la preparazione delle sessioni al giorno stesso
- Nelle 1-on-1 con i Tech Lead sono meno presente, meno curioso
- Non aggiorno il playbook da più di 6 settimane (segnale di stagnazione cognitiva)
- Non pratico Yoga da più di 2 settimane

Interventi:

Segnale | Intervento immediato

Ansia domenica sera | Bloccare giovedì pomeriggio completamente libero per la settimana successiva

Prep last-minute | Spostare prep di 1 ora al giorno precedente — non negoziabile

Presenza ridotta nelle 1-on-1 | Una settimana senza sessioni (comunicare al cliente con anticipo)

Stagnazione cognitiva | Aggiornare il playbook — il writing forzato riattiva la riflessività

No Yoga | Rimetterlo in calendario come se fosse un meeting con un cliente

Il principio NLP applicato a me stesso:

Il burnout non è un problema di forza di volontà. È un segnale che il sistema (il mio sistema di lavoro) non è calibrato. La risposta è sistemica, non disciplinare. Non mi punisco per essermi stancato — rivedo il design.

SEZIONE 8 — Template e Strumenti

8.1 Template "State of the Stack"

State of the Stack — [Nome Cliente]
Versione: 1.0 | **Data:** [YYYY-MM-DD] | **Preparato da:** Elios Scoglio

Executive Summary
[3-5 righe. Stato attuale in modo diretto. Non edulcorare, non catastrofizzare.]

1. Team

Struttura attuale
Nome	Ruolo	Seniority	Note
...	...	...	...

Punti di forza
- [...]

Gap identificati
- [...]

Rischio turnover
Persona	Livello (Alto/Medio/Basso)	Motivazione stimata
...	...	...

Raccomandazione prioritaria
[Una cosa sola, la più importante]

2. Architettura e Sistema

Stack attuale
- **Backend:** [...]
- **Frontend:** [...]
- **Database:** [...]
- **Infrastruttura:** [...]
- **CI/CD:** [...]
- **Monitoraggio:** [...]

Stato CI/CD
[Descrizione: automatico/manuale, frequenza deploy, tempo pipeline, affidabilità]

Stato osservabilità
[Descrizione: logging, metriche, alerting, tracing]

Top 5 rischi tecnici
#	Rischio	Impatto	Probabilità	Azione suggerita
1	...	Alto	Alta	...
2	...	...	...	...
3	...	...	...	...

8.2 Template Monthly Report

```
# Tech Update — [Nome Cliente]
**Periodo:** [Mese Anno] | **Data:** [YYYY-MM-DD]

---

## Highlight del mese
- [Cosa è andato bene — con risultato concreto]
- [Cosa abbiamo migliorato — con metrica se possibile]
- [Decisione importante presa — con razionale]

---

## Rischi attivi
| Rischio | Livello | Stato | Azione in corso | Owner | Scadenza |
|-----|-----|-----|-----|-----|-----|
| ... | Alto | In corso | ... | ... | ... |
| ... | Medio | Monitorato | ... | ... | ... |

---

## Decisioni richieste al CEO
1. **[Decisione]**
  - Contesto: [perché questa decisione]
  - Opzioni: [A] / [B]
  - Raccomandazione: [quale e perché]
  - Entro: [data — perché questa urgenza]

---

## Prossimo mese
- **Focus tecnico principale:** [cosa]
- **Milestone attesa:** [deliverable specifico]
- **Dipendenze critiche:** [cosa serve da chi — es. "decisione del CEO su X", "budget per Y"]

---

## Metriche chiave
| Metrica | Mese corrente | Mese precedente | Trend |
|-----|-----|-----|-----|
| Deployment frequency | ... | ... | ↑ / ↓ / → |
| Incidenti in produzione | ... | ... | ↑ / ↓ / → |
| Lead time | ... | ... | ↑ / ↓ / → |
| [Altra] | ... | ... | ... |

---

*Report preparato da Elios Scoglio — Fractional CTO*
```

8.3 Template Board Update

```
# Engineering Update — [Q1/Q2/Q3/Q4] [Anno]
**Preparato da:** Elios Scoglio, Fractional CTO
**Data presentazione:** [YYYY-MM-DD]

---

## Status Overall: [VERDE / GIALLO / ROSSO]
[Una riga di sintesi: "Il sistema è stabile e in crescita" / "Ci sono rischi attivi sotto controllo" / "Situazione criti

---

## Key Achievements del Trimestre
- **[Achievement 1]:** [risultato misurabile — es. "Ridotto il lead time da 5 giorni a 2 giorni"]
- **[Achievement 2]:** [risultato misurabile]
- **[Achievement 3]:** [risultato misurabile]

---

## Risks & Mitigations
| Risk | Business Impact | Technical Likelihood | Mitigation | Status |
|-----|-----|-----|-----|-----|
| ... | High | Medium | ... | In progress |
| ... | Medium | Low | ... | Monitored |

---

## Engineering Metrics
| Metric | Q Corrente | Q Precedente | Target |
|-----|-----|-----|-----|
| Deployment frequency | ... | ... | ... |
| System uptime | ... | ... | 99.9% |
| MTTR | ... | ... | ... |
| Team size | ... | ... | ... |

---

## Next Quarter Priorities
1. **[Priorità 1]** — Obiettivo: [risultato misurabile atteso]
2. **[Priorità 2]** — Obiettivo: [risultato misurabile atteso]
3. **[Priorità 3]** — Obiettivo: [risultato misurabile atteso]

---

## Investment Required
[Solo se serve decisione del board]
- **[Item]:** [importo/risorse] — Perché ora: [motivazione] — Impatto se non approvato: [conseguenza]
```

8.4 Template 1-on-1 Engineering Manager (mensile)

1-on-1 — [Nome] — [Data]
 Ruolo: [Tech Lead / Engineering Manager]
 Durata: 60 minuti
 Formato: video call

PREPARAZIONE (10 min prima — solo per me)
 - Azioni prese nella call precedente: [stato]
 - Cosa voglio portare io: [1-2 punti]
 - Cosa voglio ascoltare: [domande aperte]

SESSIONE

Aggiornamento (20 min)
 **Come è andato il mese? Cosa è successo?*

Note:

Sfida corrente (25 min)
 **Qual è la cosa più difficile che stai affrontando in questo momento?*

Note:

Approfondimento: **Cosa hai già provato? Cosa ti blocca?*

Note:

Supporto: **Cosa posso fare io per aiutarti su questo?*

Note:

Sviluppo (10 min)
 **Cosa stai imparando ultimamente? Dove vuoi crescere nei prossimi 3 mesi?*

Note:

Azioni (5 min)
 Lui/lei farà:
 - [] [Azione] — Entro: [data]

Io farò:
 - [] [Azione] — Entro: [data]

NOTE PRIVATE (non condividere)
 - Energia/motivazione percepita (1-5): ____

8.5 Template ADR


```

# ADR-[NNN]: [Titolo]

**Data:** [YYYY-MM-DD]
**Status:** [Proposta | Accettata | Deprecata | Sostituita da ADR-NNN]
**Deciders:** [nomi / ruoli]
**Approvato da:** [Elios Scoglio, Fractional CTO — solo per decisioni strategiche]

---

## Contesto

[Qual era la situazione? Quali vincoli tecnici, di business, o di team hanno portato a questa decisione?
Massimo 5-7 righe. Fattuale, non giustificativo.]

---

## Decisione

[Cosa abbiamo deciso. Una frase affermativa chiara.
Es: "Adottiamo PostgreSQL come database primario per il nuovo servizio ordini."]

---

## Opzioni considerate

### Opzione A: [Nome]
- **Pro:** [...]
- **Contro:** [...]

### Opzione B: [Nome]
- **Pro:** [...]
- **Contro:** [...]

### Opzione C: [Nome — se presente]
- **Pro:** [...]
- **Contro:** [...]

---

## Conseguenze

**Positive:**
- [...]

**Negative / Trade-off accettati:**
- [...]

**Rischi:**
- [...]

**Azioni conseguenti:**
- [ ] [Azione] — Owner: [chi] — Entro: [quando]

---

```

8.6 Checklist Mensile per Cliente

Checklist Mensile — [Nome Cliente] — [Mese Anno]

PREPARAZIONE (entro giorno 5 del mese)

- ☐ Leggere note del mese precedente e azioni aperte
- ☐ Controllare stato metriche (deployment, incidenti, lead time)
- ☐ Controllare issue tracker: temi ricorrenti, blocchi
- ☐ Aggiornare lista rischi
- ☐ Preparare 2-3 punti da portare allo Strategic Planning
- ☐ Bozza Monthly Report completata

SESSIONI

- ☐ Strategic Planning (2h) completato — decisioni scritte
- ☐ Architecture Review (2h) completato — note scritte
- ☐ 1-on-1 Tech Lead (1h) completato — azioni definite
- ☐ Stakeholder Update (1h) completato — report inviato

OUTPUT

- ☐ Monthly Report finalizzato e inviato al CEO
- ☐ Decisioni del mese documentate (ADR se necessario)
- ☐ Lista rischi aggiornata nel documento condiviso
- ☐ Azioni del mese con owner e scadenza scritte
- ☐ Sessioni mese successivo confermate in calendario

CONTABILITA' E ADMIN

- ☐ Ore del mese verificate (non superare time-box)
- ☐ Fattura emessa entro giorno 5 del mese successivo
- ☐ Note interne aggiornate (questo playbook se necessario)

8.7 Checklist Onboarding 50 Item (Riferimento Rapido)

PRE-ONBOARDING

1. ☐ Contratto firmato
2. ☐ Prima fattura inviata
3. ☐ Kick-off schedulato
4. ☐ Tech stack document ricevuto
5. ☐ Org chart ricevuto
6. ☐ Roadmap attuale ricevuta
7. ☐ Ultimi 3 incidenti ricevuti
8. ☐ Accesso repo (read-only)
9. ☐ Accesso issue tracker (read-only)
10. ☐ Accesso monitoring (read-only)
11. ☐ Intro email al team preparata

SETTIMANA 1 — TECH ASSESSMENT

12. ☐ Kick-off con CEO completato
13. ☐ Intro al team completata
14. ☐ 1-on-1 developer 1 completata
15. ☐ 1-on-1 developer 2 completata
16. ☐ 1-on-1 developer 3 completata
17. ☐ 1-on-1 developer 4 completata (se presente)
18. ☐ 1-on-1 developer 5 completata (se presente)
19. ☐ 1-on-1 CEO completata
20. ☐ Repository principale esplorato
21. ☐ CI/CD analizzato
22. ☐ Documentazione esistente letta
23. ☐ Dipendenze verificate
24. ☐ Secret management verificato
25. ☐ Monitoring e logging verificati
26. ☐ Note assessment settimana 1 complete

SETTIMANA 2 — ARCHITECTURE REVIEW

27. ☐ Architecture review quick completata (4h)
28. ☐ Top 5 rischi architetturali identificati
29. ☐ Top 3 opportunità identificate
30. ☐ Aree da approfondire documentate
31. ☐ Draft condiviso con Tech Lead per validazione

SETTIMANA 3 — TEAM ASSESSMENT

32. ☐ Template 1-on-1 onboarding compilato per tutti
33. ☐ Leader informale identificato
34. ☐ Rischio turnover valutato
35. ☐ Temi ricorrenti sintetizzati
36. ☐ Gap di competenza documentati
37. ☐ Potenziali Tech Lead interni identificati

SETTIMANA 4 — STRATEGIC ALIGNMENT

38. ☐ Workshop strategic alignment completato
39. ☐ Obiettivi di business 6/12 mesi documentati
40. ☐ Vincoli tecnici su obiettivi documentati
41. ☐ Lista priorità tecniche condivisa con CEO
42. ☐ Decisioni pendenti elencate con owner

OUTPUT FINALE

43. ☐ State of the Stack documento completato
44. ☐ KPI baseline stabiliti e documentati

Note Finali

Questo playbook è un documento vivo. Va aggiornato dopo ogni onboarding significativo, dopo ogni situazione difficile risolta (o sbagliata), dopo ogni insight utile.

Quando aggiornarlo:

- Dopo ogni onboarding (cosa non aveva previsto questo playbook?)
- Dopo ogni exit da un cliente (cosa avrei fatto diversamente?)
- Quando una situazione difficile richiede un approccio nuovo (aggiungerlo alla Sezione 6)
- Ogni trimestre: revisione generale, rimuovere cose che non uso, aggiungere cose nuove

Versione corrente: 1.0 — Maggio 2026

Prossima revisione: Agosto 2026

Fine documento.